Action-Bound Injective Authorization under Signer Harvesting: A Computational and Symbolic Security Analysis

Iman Schrock EMILIA Protocol, Inc. team@emiliaprotocol.ai ORCID: 0009-0004-0290-5433

Preprint. August 2026.

The construction analyzed here is also specified, in engineering form, as a set of individual Internet-Drafts (draft-schrock-ep-authorization-receipts-12 and companions). Those are individual submissions with no working-group status; this paper is self-contained and does not depend on them.

Abstract

Digital signatures authenticate messages, but signature authenticity alone does not make an authorization non-transplantable to another action or single-use at an enforcement point. We study this composition problem when the service that renders requests, routes signoffs, and retains records is untrusted. We define **Action-Bound Injective Authorization (ABIA)**: an approver-held key signs an injectively encoded context naming one exact action, while an enforcement domain admits that authorization only through an atomic consume-if-unused transition. The adversary controls the network and may obtain valid signatures on adaptively chosen contexts, modeling signer harvesting by a malicious operator. We prove that transplantation against an uncompromised approver reduces to signature forgery or a hash collision under the stated injective-encoding assumption; duplicate admission reduces to violation of the linearizable single-use state interface; and a 2-of-2 quorum reduces to those failures or violation of explicit signer-distinctness and initiator-exclusion checks. The reduction separates cryptographic assumptions from the state-service assumption and does not claim exactly-once physical effects. Four Tamarin models independently check the corresponding symbolic trace properties using linear state facts rather than a uniqueness axiom: 24 all-traces properties and five executable witnesses verify, while nine deliberately weakened comparisons produce attacks. TLA+, Alloy, implementation, and conformance artifacts validate the surrounding construction but do not replace the security argument. The analysis does not prove natural-person identity, presentation fidelity, policy correctness, log completeness, or downstream effects.

---

1. Introduction

1.1 The testimony-versus-evidence problem

Consider a dispute that is now routine in outline and will shortly be routine in fact. An AI agent operated by a financial-services firm releases a wire transfer. The transfer is later challenged, and the firm produces its approval records: a workflow-tool entry showing that a user clicked "approve" at a certain time.

Every element of that record is an assertion by the firm. The database holding the approval row is writable by the firm's administrators; the mapping between the row and the executed action is maintained by the firm's software; the timestamp is the firm's clock. An auditor, counterparty, insurer, or court that wants to rely on the record must trust the operator of the approval system, and the operator is frequently the party whose conduct is in question. The record is testimony.

Existing authorization infrastructure does not change this, because it answers a different question. Identity and access management establishes that an actor is authenticated and authorized *in general*: it grants sessions and scopes, not decisions about individual actions. Fraud that occurs inside a valid session through approved channels, such as a business-email-compromise-driven beneficiary change, is invisible to session-level controls. Where human approval does exist, it is a click recorded in a mutable store. Three gaps follow, and they are structural rather than implementation accidents:

1. **The action gap.** Authorization is granted to sessions and scopes, not to individual actions with concrete parameters.
2. **The accountability gap.** Typical approval workflows provide no independent cryptographic binding from an enrolled approver key to the canonical bytes of a specific action; the approval record is producible and alterable by the operator.
3. **The verification gap.** Those workflow records require third parties to trust the operator's logs rather than carrying a portable artifact that can be checked under independently selected trust roots.

The authorization receipt is a narrow construction aimed at exactly these gaps: before an irreversible action executes, an enrolled approver key signs a canonical context binding the exact action; the authorization reaches a terminal state at most once within the executor's shared atomic consumption domain; and the resulting receipt is verifiable offline for as long as its algorithms and independently selected trust material remain acceptable or are renewed.

1.2 The security question and boundary

The claim is deliberately limited. A receipt is evidence that a key enrolled under an approver identifier signed a canonical context binding one action, a committed policy version, and a validity window, and that the authorization reached a terminal state once within a shared atomic consumption domain. It

is not evidence that the enrolled key holder is a particular natural person, that the action was a good idea, that the approver understood the business context, that an independent executor did not accept a replay, or that the surrounding deployment cannot be bypassed. Verification proves signature, binding, and log-inclusion integrity; it never proves business correctness or a downstream effect. The organization publishing this work is not an auditor, regulator, or insurer; the artifacts described here support the judgments of such parties and conclude nothing on their own.

1.3 Security experiment

The contribution is not the observation that a signature authenticates a byte string. The question is what a relying party may infer when the party collecting signatures is itself adversarial.

Let `Enc` be the closed, injective encoding of valid typed tuples. Define the base authorization context `b = (a, p, i, r, q)`: canonical action `a`, policy commitment `p`, initiator `i`, relying-party audience `r`, and fresh shared authorization instance `q`. For approver slot `v`, let `n_v` be that slot's fresh nonce and `w_v` its validity token. The signed context is `x_v = (b, v, n_v, w_v)` and the signed message is `m_{b,v} = Enc("ep_signoff_v2", H(Enc(a)), p, i, r, q, n_v, w_v, v)`. Distinct quorum slots therefore sign distinct messages while agreeing on the same authorization instance. The domain-scoped admission key is `u_b = H(Enc("ep_admission_v1", d, H(Enc(a)), p, i, r, q))`. An enforcement domain exposes two linearizable operations: `RegisterFresh(u_b)`, which succeeds once, and `ConsumeIfUnused(u_b)`, which returns success to at most one caller and durably changes the key from `UNUSED` to `ADMITTED` before that caller may enter the provider. A timeout after provider entry is reported as `INDETERMINATE`; it is never converted into proof that the physical effect did or did not occur.

The challenger generates and pins the relying party's approver keys. For each authorization attempt, the enforcement domain issues a fresh `q`, atomically registers the corresponding `u_b`, and supplies `q` as an independently pinned input to verification; the adversarial operator may transport but not choose that value. The adversary controls the network, the operator, and the receipt presenter. It may adaptively query `Sign_j(m)` for any message under any uncompromised approver key `j`, thereby receiving genuine signatures on arbitrarily many contexts of its choice. It may reveal designated keys, but no claim is made for a key after its reveal event. It may replay, splice, reorder, or suppress every artifact and may invoke verification and admission concurrently.

The adversary wins the ABIA experiment if one of the following occurs for an unrevealed required key:

1. **Transplantation:** the verifier admits slot context `x* = (b*, v*, n*, w*)`, but the signing oracle was never queried on `m_{b*,v*}` under the pinned key that satisfied slot `v*`.

2. **Duplicate admission:** two provider-entry events in the same domain `d` are both preceded by successful consumption of the same `u_b`.
3. **Quorum substitution:** a 2-of-2 admission succeeds without two distinct pinned approver identifiers and keys, without a valid signature from each on its assigned `m_{b,v}`, without both messages sharing the same base context `b`, or with the initiator occupying either slot.

This is a protocol experiment, not a replacement for EUF-CMA signature security. A signature scheme may be unforgeable while an implementation that omits `ConsumeIfUnused`, accepts an ambiguous encoding, or leaves the initiator outside the signed context loses ABIA.

1.4 Contributions

- **A protocol abstraction and ABIA security definition.** We isolate action-bound authorization from its agent and workflow integrations, define the signer-harvesting games, terminal acceptance relation, shared consumption domain, quorum condition, and relying-party key pinning, and state the exact properties a relying party is entitled to infer.
- **A reduction with explicit non-cryptographic assumptions.** We bound transplantation and quorum forgery by EUF-CMA, hash collision resistance, and injective encoding; duplicate admission is impossible only if the named domain's consume-if-unused interface is linearizable and completely mediates provider entry.
- **Mechanized symbolic confirmation and separations.** Four Tamarin models derive the receipt and quorum properties from signature equations and linear protocol state. The strict models verify 24 all-traces properties and five executable witnesses; nine weakened comparisons produce concrete replay, initiator-binding, and stale-registry-view attacks while the signature algebra remains ideal.
- **An operator-untrusted adversary model.** The party that renders the action, routes approval, and keeps the record may drive an honest approver to sign any action chosen by the attacker. This removes the honest-renderer assumption and makes the result about protocol evidence rather than ordinary session authorization.
- **A precise construction boundary.** Canonical action bytes, approver-held keys, atomic terminal consumption, and offline verification are the analyzed core. A host-record binding profile and evidence-sufficiency graph are secondary compositions and are not claimed as new cryptographic primitives.
- **Bounded validation.** The reference implementation, 21-suite/332-vector conformance battery, and separately authored Rust verifier validate encoding and interoperability. They are not presented as independent security proofs.

The paper does not claim a new signature, hash, or transparency-log primitive. Its cryptology contribution is the ABIA composition theorem: under an

adaptive signer-harvesting adversary, exact-context authenticity and single-use admission rest on different assumptions, and omitting either binding produces an attack without weakening the signature scheme.

1.5 The result in one statement

Theorem 1 (ABIA composition). Let `Sig` be EUF-CMA secure, `H` collision resistant, and `Enc` injective on the closed authorization-context schema. Let all successful provider-entry events in domain `d` be completely mediated by a linearizable service that registers each fresh `u_b` once and later exposes `ConsumeIfUnused(u_b)`. Let verification require the signed `q` to equal the independently issued value for the attempt. For any probabilistic polynomial-time signer-harvesting adversary `A`, there exist signature forgers `B_j` and a collision finder `C_H` such that

```
Pr[ABIA_A = 1] <= sum_j Adv_euf-cma(Sig, B_j) + Adv_cr(H, C_H) +
Pr[AtomicityFail_d or MediationBypass_d].
```

For the 2-of-2 profile, the sum ranges over the two unrevealed pinned approver keys required by the accepted policy. The theorem establishes one admitted provider attempt per authorization inside `d`; it does **not** establish that a downstream physical effect occurred exactly once. It also does not extend across independent domains unless they share an equivalent linearizable admission service.

Proof. Condition on the absence of hash collisions, state-service atomicity failure, and provider-entry bypass; `Enc` is injective by construction. Suppose transplantation occurs for slot context `x* = (b*, v*, n*, w*)`. Verification succeeded under a pinned unrevealed key on `m_{b*,v*}`. If the corresponding signing oracle was never queried on `m_{b*,v*}`, the accepted signature is an EUF-CMA forgery. Otherwise an oracle query exists on the same bytes. Under injective `Enc` and collision-resistant `H`, those bytes bind the same action, policy commitment, initiator, audience, authorization instance, slot nonce, validity token, and approver slot; therefore the event is not transplantation. The locally configured domain and the shared base authorization remain bound separately into `u_b`.

For duplicate admission, both provider-entry events must, by complete mediation, follow successful `ConsumeIfUnused(u_b)` calls. Linearizability gives one total order consistent with real time. The first successful call changes `u_b` from `UNUSED` to `ADMITTED`; no transition recreates `UNUSED`, so the second call cannot also return success. Two admitted entries therefore imply either an atomicity failure or a bypass, both explicit terms in the bound.

For quorum substitution, the verifier first checks that the two slot identifiers and pinned public keys are pairwise distinct and that neither identifier equals the signed initiator. It then verifies one signature per slot on `m_{b,v_1}` and `m_{b,v_2}` and checks equality of every base-context field. If an unrevealed

slot lacks a matching oracle query, its signature yields an EUF-CMA forgery. If both queries exist, injectivity and collision resistance make each signed slot context equal to the one the verifier accepted, while the explicit base-context equality check makes both signatures authorize the same action, policy, initiator, audience, and authorization instance. Each slot's nonce and validity token remain separately bound to that slot. The explicit distinctness checks exclude one signer or the initiator from filling both roles. This exhausts the winning conditions. A union bound over the bad events gives the stated inequality. QED.

The mechanized result in Section 7 is independent corroboration of this argument in the Dolev–Yao trace model. Its checked acceptance rule consumes a linear `AdmissionAvailable` fact created for the fresh context; it does not use a restriction asserting that consumption is unique. Removing that state input produces the expected same-receipt replay. Removing the initiator from the quorum signature or omitting the exact pinned registry view produces the other reported separation traces.

2. Threat Model

Three adversarial settings drive the design. They are treated together because a mechanism that addresses one while silently assuming away another produces evidence that fails exactly when it matters.

The adversary here is stronger than the one standard authorization models carry, and the difference is the point of the work. Session and delegation systems, from OAuth scopes and Rich Authorization Requests through transaction tokens and delegation receipts, reason about which principal may act, and they assume that an honest principal signs only what a correct protocol run presents to it. This setting removes that assumption. The operator that renders the action to the approver, routes the approval, and later holds the record is untrusted for evidence purposes, and it can present an honest human with any action it chooses to have signed. The question is therefore not whether the actor was authorized in general, which the delegation layer answers, but whether an accepted record can still prove that an uncompromised key enrolled to the cited approver identifier signed a context binding exactly the accepted action when the party producing the record is the one whose conduct is later in dispute. The symbolic analysis in Section 7 is run against exactly this adversary, which is what makes its authenticity and no-replay results say something the delegation layer cannot.

2.1 Compromised or malicious operator

The party running the orchestration service (policy registry, signoff routing, log) is not trusted for evidence purposes. Under the recommended key custody classes and the model's uncompromised-key assumption (Section 4.4), a compromised operator can deny service and can fail to route signoff requests; it cannot

forge a signature under an enrolled approver key it does not hold. Within a shared atomic consumption domain, a second presentation of a genuine authorization is rejected. Independent executors require a shared consumption service or equivalent coordination to obtain that replay guarantee.

Two operator-compromise paths remain open and are part of the model rather than claimed away. First, an operator that controls the signing client's rendering can harvest a *genuine* signature over an action the approver misunderstood; this is the presentation-attack family of Section 10.2, and it is why independently authored rendering surfaces are required for high-value policies. Second, an operator that unilaterally controls the directory of enrolled approver keys can enroll a key it controls under a legitimate approver's name, relocating the forgery rather than preventing it; Section 4.4 describes the directory-authority controls. The accurate statement of the modeled property is therefore: under the uncompromised-key assumption, the operator cannot forge a signature under an enrolled approver key. The stronger statement, that the operator cannot obtain an unauthorized approval at all, additionally requires the directory-authority and independent-rendering controls to be deployed.

2.2 Compromised agent

The initiating agent is identified but never trusted. A prompt-injected or otherwise compromised agent can propose any action, state any escalation reason, and attribute itself to any external identity. The design consequence is uniform: everything the initiator supplies is treated as a claim. Injection can change what the initiator *proposes*; it cannot change what a human *approves* on the human's own hardware, because the device-bound signature is produced outside the model's context. The protocol additionally enforces separation of duties: an initiator can never occupy an approver slot for its own action. This separation is one of the properties the symbolic quorum model verifies (Section 7).

2.3 Post-hoc dispute

After execution, the parties evaluating the evidence (auditors, counterparties, insurers, courts, regulators) may distrust every online service involved, may be examining events years old, and may have no network path to the original operator, which may no longer exist. The design consequence is offline verifiability: a receipt must be checkable using only its own bytes, pinned or directory-proven approver key material, and a log checkpoint carried inside the receipt. What offline verification can and cannot establish in this setting is bounded precisely in Sections 4.3 and 10.3.

2.4 Out of scope

The protocol binds approvals to *approver identifiers* whose keys are enrolled in a directory; proving that the holder of an identifier is a particular natural person is the job of the identity-proofing layer that populates the directory, not

of the receipt format. Collusion among distinct enrolled humans, one human controlling multiple enrolled identities, and coercion of an approver are likewise not defeated by the mechanism; receipts make such events attributable, which raises their cost, and no more than that. The symbolic analysis is explicit about this boundary: it proves the distinct-identity and no-self-approval property, and nothing about whether two distinct identities are two independent wills (Section 7.4).

3. The Receipt Construction

3.1 Actions as canonical bytes

An action is a single proposed operation with concrete parameters: one wire transfer to one beneficiary for one amount, one credential rotation of one key. It is represented as a JSON Action Object containing at minimum the action type, target system and resource, parameters, initiator identity, policy reference, and request time. The object is serialized under the JSON Canonicalization Scheme (JCS, RFC 8785), and the *action hash* is the SHA-256 digest of the canonical serialization. Implementations must reject an approval request whose action hash does not match a locally recomputed hash of the presented object. Sensitive parameter values may be carried as salted hashes, provided the executing system can recompute them; the binding property is preserved because the hash commits to the committed values.

Canonical bytes are the foundation of every later property: an approval bound to a description or a workflow-ticket identifier binds to something the operator can re-narrate; an approval bound to the SHA-256 of canonical bytes binds to exactly one action. The symbolic models of Section 7 make this abstract but load-bearing: they treat message encoding as injective (the standard symbolic assumption) and derive no-replay-across-actions from the fact that each signature is over the action's hash, so the attacker cannot repurpose a signature over one action term as a signature over another.

3.2 The authorization context and the signoff

For each authorization attempt, the relying party, authorization server, or enforcement domain first issues and durably registers a shared authorization instance of at least 128 bits of CSPRNG output. The untrusted orchestrator receives that value but does not select it. For each required approver, the orchestrator then constructs an Authorization Context: a canonical JSON structure containing the action hash, the policy identifier *and* a policy hash committing to the exact policy version evaluated, the initiator identity, the shared authorization instance, the approver identifier and slot index, the required approval count, a distinct per-approver nonce of at least 128 bits of CSPRNG output, issuance and expiry times, and a hash chaining the attempt to the issuing log's most recent receipt. Every context in one approval ceremony signs the same authorization instance; each context signs a different nonce. The context is

JCS-canonicalized; the *context hash* is its SHA-256 digest; the approver signs the context hash.

Four bindings deserve emphasis, and each is exactly the kind of load-bearing detail a symbolic prover exposes. First, the policy commitment is exact: a signature over a context with policy hash X must not satisfy a requirement evaluated under policy hash Y, even for the same policy identifier. Approvals cannot migrate across policy versions. Second, the initiator identity is bound *inside* the signed context, not merely tracked as an executor-local label; Section 7.3 records that an earlier quorum model that omitted this binding was falsified, because separation of duties then held only at the executor and not in the signatures themselves. Third, the shared authorization instance binds all quorum signoffs to one approval ceremony. Without it, an adversarial collector can splice individually valid signoffs from different ceremonies whose action, policy, audience, and approver slots happen to match. Fourth, the approver must be shown, at signing time, a faithful human-readable rendering of the Action Object, not only its hash; interfaces that display a different action than the one hashed constitute a presentation attack (Section 10.2).

The context may optionally carry two further members, both claims by parties the protocol identifies but never trusts. An *initiator attestation* records the initiator's own stated reason for escalating to a human, as a closed enumeration (irreversibility, magnitude, uncertainty, novelty, authority gap, policy rule) plus an optional rule identifier and a length-capped free-text statement. An *agent binding* attributes the action to an external agent identity and, optionally, the external delegation under which the agent acted, with an optional observed-at timestamp against which a relying party may enforce evidence freshness, fail-closed. Because JCS serializes every member present, both are covered by the approver's signature: the receipt proves the stated reason and claimed attribution were part of what the approver signed. Neither is proof of the initiator's internal state, the agent's identity, or the delegation's validity, and policy engines must not use attestation content to relax any threshold: the initiator must gain nothing by saying the right words.

The signoff itself carries the context hash, the signature, the key custody class, the approver key identifier, and, for device-bound keys, the WebAuthn assertion material. For device-bound keys the WebAuthn challenge must equal the context hash and the authenticator's user-verification flag must be set, so the signature attests both possession of the enrolled authenticator and a local user-verification event. In the symbolic models this user-verification requirement is modeled structurally as a linear precondition that only a user-verification step can satisfy, so no trace contains a signature without a preceding user-verification event over the complete signed authorization context (Section 7.2); the model assumes the authenticator enforces user verification and does not itself prove WebAuthn internals. Denials are signed over the same context hash with a denial envelope, so refusals are equally attributable to the enrolled key and equally terminal: "the enrolled approver key signed a refusal of this action" is a positive,

verifiable fact, not an absence.

3.3 Core protocol interface and adversarial queries

The cryptographic core has four roles: approver key holders, an operator that may be malicious, a relying party, and an enforcement domain `D`. The operator constructs one base context `b` around the independently issued `q` and one slot context `x_v=(b,v,n_v,w_v)` for each required approver. The relying party accepts only closed, canonically encoded contexts whose base fields are byte-identical, verifies each signature under independently pinned key material, enforces signer/slot distinctness and initiator exclusion, derives `u_b`, and invokes the atomic consume operation before provider entry. A receipt may include a transparency-log inclusion proof, but that proof authenticates publication; it does not replace either the signature checks or the consume operation.

The following algorithms define the analyzed 2-of-2 profile; the one-signer profile is the same construction with one slot:

```
Issue_D(a, p, i, r):
    repeat q <- CSPRNG(lambda)
    until RegisterFresh_D(H(Enc("ep_admission_v1", D, H(Enc(a)), p, i, r, q)))
    return q

Sign(sk_j, b, v_j, n_j, w_j):
    m_j <- Enc("ep_signoff_v2", H(Enc(a)), p, i, r, q, n_j, w_j, v_j)
    return (v_j, n_j, w_j, Sign.S(sk_j, m_j))

VerifyContext(pk_j, b, v_j, n_j, w_j, sigma_j):
    reject unless b, v_j, n_j, and w_j parse under the closed schema
    m_j <- Enc("ep_signoff_v2", H(Enc(a)), p, i, r, q, n_j, w_j, v_j)
    return Sign.V(pk_j, m_j, sigma_j)

VerifyAndConsume_D(b, {(id_j, pk_j, v_j, n_j, w_j, sigma_j)}_{j=1,2}):
    reject unless the two entries have the same b
    reject unless q equals D's independently issued value for this attempt
    reject unless q is shared and n_1 != n_2
    reject unless ids, keys, and slots are pairwise distinct
    reject if either id equals initiator i
    reject unless VerifyContext succeeds for both pinned keys
    u_b <- H(Enc("ep_admission_v1", D, H(Enc(a)), p, i, r, q))
    reject unless ConsumeIfUnused_D(u_b) atomically returns success
    emit ProviderEntry(D, u_b, a)
```

`RegisterFresh_D` and `ConsumeIfUnused_D` are stateful interfaces, not cryptographic primitives. Their contract is linearizability, no recreation of an existing admission key, durable transition from `UNUSED` to `ADMITTED`, and complete mediation of every `ProviderEntry` in `D`. The reduction names a failure of that

contract explicitly instead of treating it as a signature failure.

The formal adversary is allowed to call the honest signing rule on arbitrary actions and initiators. This models an operator that can make a genuine signing request look legitimate to the signing client. The models do not grant the adversary the signing key: `RevealLtk` is an explicit compromise event, and each authenticity lemma is conditioned on its absence before acceptance. This separation is important. The theorem addresses misuse of valid signatures; it does not claim resilience after key extraction or against a compromised trusted display path.

3.4 One-time consumption as enforcement-point state

An authorization attempt moves through a small state machine:

```
REQUESTED -> {PARTIALLY_APPROVED}* -> APPROVED -> COMMITTED
          \-> DENIED                           \-> EXPIRED
          \-> EXPIRED
```

COMMITTED, DENIED, and EXPIRED are terminal. The construction's invariants, maintained as machine-checked models (Section 7) and required of conforming implementations, are:

- **ConsumeOnce.** Within one shared atomic consumption domain, an authorization instance transitions to a terminal state at most once; any second presentation to that domain is rejected as a replay. A legacy single-context receipt without the shared instance uses its context nonce as the instance key.
- **BindingMatch.** A signoff satisfies only the context, and therefore only the action hash, that it signs.
- **TerminalIrreversibility.** No transition exits a terminal state.
- **SelfApprovalImpossible.** For every signoff, the approver differs from the initiator; under multi-approver policies, approvers are pairwise distinct and each distinct from the initiator.
- **NoBypassWrite.** A COMMITTED state is reachable only through the full sequence, and a conforming verifying executor does not execute without verifying it.

The consumption record is enforcement-point state, not log decoration, and its necessity is not merely asserted: the symbolic core model (Section 7.2) removes the consumption check and the prover immediately falsifies injective acceptance, producing a same-receipt replay trace with no forgery and no key compromise. Adding the check back restores injectivity. Within the shared atomic consumption domain, an authorization, once consumed or refused, is terminally unusable; a later presentation to that domain is detectable and must be rejected. Independent executors obtain the same guarantee only if they coordinate through that domain or an equivalent action-keyed atomic admission service. This separates an admitted authorization from a reusable bearer credential, but it does not by

itself prove that a downstream external system produced exactly one effect.

Deployment topology is graded honestly rather than assumed. In the strongest conformance class, the system of record itself (payment switch, registry, deployment controller) verifies the authorization bundle before executing and refuses otherwise. A middleware class intercepts between the agent and the executing credential, which defends against agent error and prompt injection but can be bypassed by an operator with code control. An evidence-only class makes no enforcement claim at all. Implementations must declare their class in the receipts they produce and must not state a stronger class than deployed; the distance between "the construction has these properties" and "your deployment is unbypassable" is the most common overclaim in this category.

3.5 The trust receipt and offline verification

Upon commitment, the orchestrator assembles the Trust Receipt: the full Action Object and its hash, every authorization context, every signoff, the consumption record, a Merkle inclusion proof for the receipt leaf against a signed log checkpoint (tree size, root hash, log signature, log key identifier), and inclusion proofs for the approver key entries in the approver directory. Operator-produced commitments and receipt-log material are signed with Ed25519; approver signoffs under the device-bound class are ES256 (P-256) or Ed25519 where the authenticator supports it.

A verifier holding the receipt, a trusted log public key, and a trusted directory root or pinned approver keys, with *no network access*, establishes six things: (1) the action hash recomputes from the canonical Action Object; (2) each context hash recomputes and commits to the action hash, the policy hash, and a distinct approver; (3) each signature, including the WebAuthn assertion where present, verifies over the context hash against a key entry whose validity window contains the issuance time; (4) separation of duties holds and the approval count satisfies the policy; (5) the Merkle inclusion proof verifies against the checkpoint root and the checkpoint signature verifies against the log key; (6) signing and commitment times fall within the validity window.

Step (5) is what distinguishes this design from log-access designs: the checkpoint travels inside the receipt, so verification requires no query to the log.

One further property is deliberate and load-bearing: no step of offline verification relies on a message authentication code, a shared secret, or any symmetric primitive. A symmetric construction, such as an HMAC-chained audit log, is verifiable only by a party holding the same secret as the producer, precisely the party the evidence is meant to constrain; its keeper can rewrite its own history undetectably. Every artifact on the verification path is bound by an asymmetric signature whose verifying key the relying party holds independently of the issuer.

Equally important is what offline verification does *not* establish, and the spec-

ification requires implementations to say so. It establishes authenticity as of commit time, not currency: a receipt whose approver key was revoked an hour after commitment still verifies, because the artifact is evidence of validity at commit time. And it establishes inclusion in *a* tree whose head the log operator signed, not that this tree is the only tree the operator has shown the world; split-view detection requires online activity such as gossip or witness cosigning. A relying party with freshness or revocation requirements must additionally consult a current directory head and checkpoint online. Implementations must not describe offline verification as establishing that a receipt is "currently valid."

3.6 Key custody and the approver directory

Key classes classify custody of the approver's signing key and nothing else. Class A keys are generated and held in a platform authenticator or security key and exercised via WebAuthn with user verification required; this is the recommended class and the one the symbolic models take as their setting. Class B keys are software keys in the approver's client environment, acceptable where WebAuthn is impractical, with the reduced assurance noted in receipts. Class C, in which the operator signs on the approver's behalf after authenticating them, exists only to describe pre-existing deployments; receipts produced under it must be labeled as such, and relying parties should treat them as operator assertion, not approver signature.

Approver public keys are enrolled in a signed, Merkle-tree-structured approver directory maintained per organization, and a receipt carries an inclusion proof of the relevant key entry, so verification needs no live directory access. Directory authority is itself a trust root and must not default to the operator: the directory tree head must be signed by an organization-controlled key, and where directory *operation* is delegated to the operator, every enrollment must carry a second-party attestation by an organization administrator key or a quorum of already-enrolled approvers. A verifier presented with a directory head signed only by an operator-held key treats the whole structure as operator assertion, equivalent in assurance to Class C, regardless of the custody class of individual signoffs. The rationale is the relocation argument of Section 2.1: an operator that cannot forge signatures but can decide which keys count has not been removed from the trust path. The symbolic models abstract this directory as a single out-of-band key-pinning step; modeling the directory protocol and its trust root is stated future work (Section 7.4), not a proven property.

The directory is also the explicit slot where identifier-to-person binding lives. The receipt format proves that a key enrolled under a given identifier signed the exact action; the strength of the claim that the identifier names a particular human is a property of the directory authority and whatever identity-proofing layer feeds it, and it is deliberately not smuggled into the receipt's own claims.

4. Quorum and Distinctness

High-consequence actions frequently require more than one approver, and multi-party approval introduces a failure mode that single-approver analysis misses: counting signatures instead of humans. This is the section the symbolic quorum model of Section 7.3 analyzes directly.

Under an m-of-n policy, each required approver receives and signs an individual authorization context sharing the same action hash and authorization instance but carrying a distinct approver index and nonce. Commitment occurs only when the required number of valid, distinct signoffs exists before expiry. Partial approval confers no authority whatsoever: a verifying executor presented with fewer than the required signoffs refuses. The symbolic model encodes this literally: it has no accept rule that fires on a single signature, and its commitment rule requires both signatures to carry the same authorization instance.

The distinctness requirement is stated at the level the mechanism can enforce: the required number of *distinct enrolled approver identities*, with distinct keys, not merely distinct signature instances. The corresponding machine-checked properties include, in the state-machine models, SelfApprovalImpossible together with the properties that no enrolled identity fills two quorum slots and no key fills two quorum slots, and, in the symbolic quorum model, that a 2-of-2 commitment requires signatures under two distinct enrolled identities over the same action bound to the same initiator, that the initiator cannot self-approve, and that no single signer fills the quorum (Section 7.3). Together these rule out one enrolled identity filling two slots and one credential being presented twice. They do not prove that the identities correspond to independent natural persons; that depends on the directory's identity-proofing controls. Where a policy requires ordered approval, the chain is checked for acyclicity and linearity.

One residual vector in the multi-approver setting survives all signature checking and is stated plainly. Since each approver signs their own context, a malicious orchestrator can show different approvers different optional-member content (for example, different initiator attestations), and every individual signature remains valid. The specification therefore requires cross-context consistency: an optional member present in any context of a receipt must be present, byte-identical in canonical form, in every context, so every approver demonstrably signed the same stated reason and attribution. Verifiers implementing the members flag violations; the rule is a conformance check layered above signature validity, not a change to it.

Delegated approval authority is constrained rather than prohibited: a delegation record must appear in the receipt's key proofs, a delegate's effective scope is the intersection of the delegation grant and the principal's authority at signing time, chains are bounded in depth, and every link is independently signed.

5. Host-Record Binding

Receipts do not live alone. The agent-action record formats now in development (post-execution capsules, pre-execution permits, audit-trail records, provenance graphs, intent chains) almost uniformly make the same, sound scoping decision: human authorization is somebody else's format. The result is a set of reserved-but-empty slots: an approver disposition whose authority is opaque, an authority-context stub, a human-override field with a privacy carve-out where the human should be, a "signed grant" whose format is an unassigned work item. A dozen slots with a dozen ad-hoc answers would be worse than none. The binding profile is composition *around* the analyzed core of Sections 3 and 4, not a second protocol: a bound artifact verifies under its own specification, and the receipt whose bytes it references is the one the symbolic models are about.

The binding profile defines, host-agnostically, what filling such a slot means. Evidence is bound either **by reference** (a member carrying the SHA-256 digest of the authorization artifact's canonical bytes, an artifact-type token, and an optional locator hint) or **embedded** (a compact self-describing claim carrying the action digest, mode, named approvals with each approver's own signature object, and optional quorum semantics). Five requirements make the binding mean the same thing in every host:

- **B1, digest grounding.** A binding is credited only against artifact bytes: the reference form by digest equality, the embedded form by verifying the contained signatures. A host field that merely asserts "a human approved," with neither, must not be treated as human-authorization evidence.
- **B2, action agreement.** When the host record binds an action, the authorization artifact's action binding must agree with it. An artifact authorizing a different action invalidates the binding; it does not merely weaken it. This defeats splicing a genuine artifact from one action into another's record, the confused-deputy case.
- **B3, verified versus accepted.** Verifying a binding (digests and signatures hold) is distinct from accepting it (the relying party trusts the artifact's issuer via out-of-band pinned key material). A verifier must report the two separately, and a binding from an unpinned issuer must not be accepted; a self-issued artifact cannot be self-accepted.
- **B4, fail-closed absence.** The absence of a binding is the absence of evidence, never a default. A relying party whose policy requires human authorization treats an unbound or unresolvable binding as insufficient. Absence becomes positive evidence only through a signed *observed-absence* statement: an attestation that the verifier performed a defined search against defined sources at a stated time and found none, attesting the search and its emptiness, never a universal negative.
- **B5, consistency.** When a host record carries both forms, the embedded claim's canonical bytes must hash to the reference's digest; a mismatch invalidates both. The two forms may not tell two stories.

For transparency-logged signed statements (the SCITT host family, RFC 9943), the profile fixes the digest choice per host-statement profile: either a digest over the receipt payload's canonical bytes, for offline composition, or a digest over the COSE_Sign1 bytes of the receipt expressed as a signed statement, recommended when the receipt is registered in a transparency service so the reference also resolves to a logged, inclusion-proofed entry. The reference form additionally serves disclosure minimization: the approver identifier travels only in the artifact, which can be withheld until a relying party with a need to know requires it, at the cost, by B4, of the evidence not counting until produced.

Two boundary rules complete the profile. A host record must not extend an artifact's validity by carrying it; replay and one-time-use semantics belong to the artifact and its enforcement point. And the profile defines no authorization format of its own: a bound artifact verifies under its own specification.

6. Evidence Sufficiency: the Action Evidence Graph

6.1 The relying party's question

The standards landscape now produces many signed artifacts about one agent action: workload identity credentials, delegation and grant tokens, call-chain transaction tokens, runtime attestation results, pre-execution permits, action-bound human-approval receipts, post-execution records, transparency-log inclusion receipts. Each answers its own question. None answers the relying party's: given all of these artifacts, is this action's evidence sufficient *for this reliance purpose*? A bank releasing a wire, an insurer paying a claim, and a regulator auditing an agent need different sufficiency bars over the same artifacts, and today each such decision is ad hoc, non-reproducible, and leaves no evidence of its own. As with the binding profile, this layer composes *around* the analyzed core: it consumes the receipt as one node among several and inherits, without strengthening, the trust model of each artifact it references.

6.2 Content-addressed graphs, edges as claims

An Action Evidence Graph is a portable JSON document carrying the action digest, nodes, and edges. Each node's identifier is the SHA-256 digest of the referenced artifact's JCS-canonical bytes, with a type token and, optionally, the artifact inline; a verifier rejects an inline artifact whose canonical digest does not equal its node id. Because nodes are references, a presenter can disclose the *shape* of its evidence without the contents; an undisclosed node contributes nothing to sufficiency, and a required undisclosed node fails closed.

Edges are presenter claims, and this framing carries the security weight of the layer. An edge (for example, "this permit *permits* that authorization receipt") is credited only if the claimed binding is present in the source artifact's own canonical bytes, at minimum containment of the target's digest. A claimed edge that the bytes do not back *poisons* the evaluation: the graph has asserted something its evidence does not support, and the verdict must not be admissible.

Graph identity is disclosure-independent: the graph digest is computed over the sorted (id, type) node pairs, the sorted edges, and the action digest, and never over inline artifact bytes, so two parties holding different disclosures of the same graph agree on what graph they are discussing.

6.3 Policy replay and the closed verdict set

The sufficiency bar is an evidence policy supplied by the relying party. The graph document has no policy member, by construction: a presenter choosing its own sufficiency bar is the confused-deputy failure of evidence systems. A policy names its reliance purpose, a boolean requirement expression over node types, per-type freshness bounds, per-type revocation requirements, and required edges that must be present and byte-backed.

Evaluation is deterministic and offline: given the same graph, policy, and evaluation time, any implementation reaches the same verdict and the same replay digest. The verdict is one of a *closed* set of five: `admissible`, `missing_evidence`, `stale`, `conflicted`, `unverifiable`, with precedence **unverifiable over conflicted over stale over missing_evidence over admissible**, so no failure ever degrades toward admissibility. Machine-readable reason codes accompany the verdict; the reason vocabulary is open, the verdict set is not.

The failure semantics encode a deliberate distinction between absence and deception. A node referenced but not disclosed contributes an unverified fact; if the policy required that type, the verdict is `missing_evidence`: the evidence may exist but was not presented, and nothing about the graph lied. A claimed edge the source bytes do not back is a lie about the evidence and forces `unverifiable` regardless of what else verified. Absence degrades to "not enough"; deception degrades to "not trustable."

The replay digest is the canonical digest of exactly three inputs: the policy as supplied, the ordered list of normalized evidence facts (per node: type, label, verified flag, attested action digest, outcome if any, revocation flag, age, derived staleness, and whether a revocation check was required), and the evaluation time, bound to the graph digest. Raw artifact bytes, network state, and clock reads are excluded by construction; two evaluators that disagree on a replay digest are, by definition, evaluating different inputs.

6.4 Reliance results and policy packs

The verdict may itself be issued as a signed artifact carrying the verdict, reasons, action digest, graph digest, policy digest and identifier, reliance purpose, replay digest, and evaluation time, signed by the relying party's verifier key. The signature adds accountability, never authority: any third party can recompute the replay digest rather than trust the signer, and the verified/accepted distinction of B3 applies to the result itself. Signed reliance results make reliance decisions auditable: an institution can prove, later, which policy it applied to which evidence before it acted.

Six policy packs profile the mechanism for concrete irreversible action classes: wire transfer, vendor bank-account change (a distinct-human quorum), credential rotation, production data deletion, regulated trade execution (post hoc: the execution record must provably reference its authorization and be transparency-logged), and healthcare record export. A pack is a starting point the relying party adopts and owns, pinning its own trust anchors and tightening freshness to its risk appetite; no pack waives fail-closed behavior.

A verdict of `admissible` is evidence that a bundle met a stated policy at a stated time. It is not adjudication, it does not establish that the action was correct or safe, and it confers no authority beyond what the underlying artifacts carry. The token is a protocol token naming sufficiency under the stated policy; it carries no legal meaning.

7. Formal Analysis

The reduction in Section 1.5 is the security argument. This section provides an independent symbolic check, under a Dolev-Yao attacker, of the receipt core, a 2-of-2 quorum instance, and two composed acceptance models spanning authority, registry view, reliance configuration, challenge consumption, and fleet execution. We present that analysis first (Sections 7.1 through 7.4), then the complementary state-machine and relational model checking that surrounds it (Section 7.5), and finally the discipline that keeps *specified* separate from *verified* (Section 7.6).

7.1 The symbolic model and its attacker

The four Tamarin models are the EP models in which "the signature verifies" is a *derived* fact rather than an axiom. Signatures are terms in Tamarin's standard `signing` equational theory, `verify(sign(m, k), m, pk(k)) = true`, and the adversary is the standard Dolev-Yao network attacker: full control of the network, sees every message, may apply all function symbols, may extract a device key through an explicit compromise rule, and may additionally request an honest human to sign arbitrary actions of the attacker's own choosing. The attacker's ability to collect honest signatures over any actions it likes is what makes the no-replay-across-actions results meaningful: the prover must rule out repurposing any of those honest signatures onto a different action.

The interesting part is the adversary, not the tool. A symbolic analysis usually lets honest principals sign only what a correct protocol run dictates. Here the party running the protocol is untrusted for evidence purposes, and it can drive an honest human to sign any action it chooses, which is the realistic shape of an operator that controls both the signing client and the approval workflow and later has an interest in the record. The models ask whether, against that operator-untrusted, signature-harvesting adversary, an accepted receipt still implies a prior user-verification-gated signature under the pinned enrolled key over exactly the accepted action, whether a satisfied quorum still requires two dis-

tinct enrolled identities rather than one signature counted twice, and whether the one-time-consumption and initiator bindings are what carry those properties. The models answer yes under their stated abstractions, and dropping either binding produces a counterexample. They do not prove natural-person identity, independent human judgment, WebAuthn internals, or downstream exactly-once effects. The primitives are standard; the contribution is the security argument about them under this adversary, which is the adversary that the evidence-versus-testimony problem of Section 1 actually presents.

The models were checked with tamarin-prover 1.10.0 (Maude 3.4), all well-formedness checks passing. `ep_receipt_core.spthy` analyzes a single user-verified signature, its acceptance, and one-time consumption. `ep_quorum_core.spthy` layers a 2-of-2 quorum on top of the same user-verification-gated signature discipline. `ep_reliance_composed.spthy` re-derives an abstract 2-of-2 approval path together with challenge, action identifier, profile and audience, initiator, authority, exact registry view, revocation, issuer pinning, consumption, and execution. `ep_six_claim_composed.spthy` focuses on Class-A downgrade refusal, denial non-authorization, scoped authority, relying-party-pinned profiles, registered challenge consumption, and canonical action-keyed fleet execution. The models do not cover full WebAuthn internals, directory publication, or transparency-log mechanics; Section 7.4 states the exact composition and exclusions.

7.2 Receipt core: verified lemmas and the consumption falsification

The core model maps to the receipt construction of Section 3. A human approver holds a device-bound key whose public half is published (so the attacker always knows it). User verification is modeled structurally: the signing rule can consume only a linear fact that the user-verification step alone produces, so no trace contains a human signature without a prior user-verification event over the same action hash, policy commitment, initiator, audience, nonce, validity token, and approver slot. The relying party pins the human's key out of band (abstracting the approver directory as a single step) and accepts a receipt only if the signature verifies under that pinned key over the complete abstract context. The request rule creates one linear `AdmissionAvailable` fact for the fresh context. The checked acceptance rule must consume that fact; the unchecked comparison rule omits it. There is no uniqueness restriction that states the desired result.

The machine-checked result, verbatim from the run, was:

```
executable_honest_receipt (exists-trace): verified (9 steps)
core_authenticity_uv_gated (all-traces): verified (11 steps)
no_replay_across_actions (all-traces): verified (11 steps)
injective_acceptance_with_consumption (all-traces): verified (11 steps)
unchecked_acceptance_is_injective (all-traces): falsified - found trace (11 steps)
```

What each verified lemma establishes:

- `executable_honest_receipt` (verified). A sanity trace exists: an honest, user-verified, consumption-checked receipt is accepted with no key compromise, so the model is not vacuous.
- `core_authenticity_uv_gated` (verified, authenticity gated on user verification). If any relying party accepts a receipt naming human H for one complete abstract context, and H's key was not compromised before acceptance, then H performed a user-verification event and signed that same context, in that order, before acceptance.
- `no_replay_across_actions` (verified, no transplant across contexts). No trace accepts a context that the uncompromised human did not sign. The attacker holds honest signatures over arbitrary other contexts of its choice and still cannot transplant any of them by changing the action, policy, initiator, audience, nonce, validity token, or approver slot.
- `injective_acceptance_with_consumption` (verified, injective acceptance under one-time admission). Each checked acceptance corresponds to a preceding honest signature, and the linear availability fact prevents a second checked acceptance of that same complete context.

The fifth lemma, `unchecked_acceptance_is_injective`, is falsified *by design*. It asserts injective acceptance without consuming state. Tamarin produces the expected trace: the attacker takes the honest receipt broadcast by the signing rule and delivers it to an unchecked accept point twice. No forgery and no key reveal occur. Requiring the checked rule to consume the sole linear availability fact removes that trace. The difference between the models is therefore an executable state transition, not an axiom that duplicate consumption cannot occur. This establishes the symbolic half of the assumption split in Theorem 1: signature verification carries exact-context authenticity, while linear state carries single-use admission.

7.3 Quorum: verified lemmas and the initiator-binding falsification

The quorum model maps to Section 4. It enrolls two distinct approver identities, each with its own device-bound key; a restriction enforces that a single name cannot be enrolled twice, so the two quorum slots are two distinct names. An initiator identity, chosen by the attacker, proposes the action. One request rule creates a fresh shared authorization instance and a distinct nonce for each approver. The initiator and authorization instance are bound *into* each signed context, abstracted as `<'ep_signoff_v1', h(action), initiator, authorization_instance, nonce>`. Each approver signs its own context, user-verification-gated as in the core model. Commitment fires only when two signatures verify under two distinct pinned approver keys over the same action, initiator, and authorization instance, and neither approver is the initiator; there is no accept rule that fires on a single signature or combines signatures from separate ceremonies.

The machine-checked result, verbatim from the run, was:

```
executable_quorum (exists-trace): verified (13 steps)
quorum_requires_two_distinct_uv_gated_signatures (all-traces): verified (27 steps)
initiator_cannot_self_approve (all-traces): verified (4 steps)
no_single_signer_fills_quorum (all-traces): verified (4 steps)
commit_requires_signature_over_that_action (all-traces): verified (8 steps)
```

What each verified lemma establishes:

- `executable_quorum` (verified). A 2-of-2 quorum can commit with two distinct honest approvers, both user-verified, neither the initiator, no key compromise, so the distinctness and self-approval restrictions do not make commitment unreachable.
- `quorum_requires_two_distinct_uv_gated_signatures` (verified). Whenever the executor commits action a and authorization instance q citing approvers H1 and H2, and neither key was compromised before the commit, H1 and H2 are distinct and each performed user verification followed by a signature over exactly (a, q), all before the commit. A single signature, two signatures from one identity, signatures over different actions, or signoffs from different authorization instances can never commit.
- `initiator_cannot_self_approve` (verified). No trace commits an action with the initiator itself appearing as either quorum approver. Because the initiator identity is attacker-chosen, this also rules out an attacker naming a compliant approver as initiator to have it count against itself.
- `no_single_signer_fills_quorum` (verified). The two committing approvers are never the same identity; distinct pinned keys entail distinct enrolled identities. This is the no-key-fills-two-slots property at the identity level.
- `commit_requires_signature_over_that_action` (verified). No trace commits (a, q) while an uncompromised named approver never signed exactly (a, q). Because the attacker can obtain honest signoffs over arbitrary other actions and ceremonies, this rules out transplanting a signature from either.

As with the consumption check, an earlier revision of this model was falsified, and the record is kept because it changed the construction. In that first revision the signed authorization context did *not* bind the initiator identity; it was `<'ep_signoff_v1', h(action), nonce>`. Under that revision Tamarin falsified both `quorum_requires_two_distinct_uv_gated_signatures` and `commit_requires_signature_over_that_action`: two honest approvers signed action a while the request carried one initiator label, and the executor committed the same a under a *different* initiator label, because nothing tied the approver signatures to the committing initiator. Separation of duties was only an executor-local check, not a property of the signatures. The fix is spec-faithful and not a lemma weakening: the initiator identity is now bound inside what each approver signs, and with that binding all five lemmas verify. The falsification identified a load-bearing binding, and the model now shows

that binding is what carries the property. This mirrors the core model's consumption result exactly: a check the design depends on is shown to be depended upon.

7.4 Composed acceptance paths and the exact scope

The third model, `ep_reliance_composed.spthy`, puts the following in one symbolic trace: a signed reliance challenge; a computed action identifier binding family and material fields; exact profile, audience, and initiator binding; two distinct user-verification-gated approvals; scoped authority under an exact pinned registry epoch and head; revocation state; a pinned receipt issuer; one-time consumption; and execution. Its machine-checked summary is:

```
executable_composed_reliance (exists-trace): verified (20 steps)
execution_requires_full_composition (all-traces): verified (101 steps)
caid_binds_family_and_material (all-traces): verified (2 steps)
initiator_cannot_self_approve (all-traces): verified (4 steps)
no_single_signer_fills_quorum (all-traces): verified (2 steps)
no_issuer_laundering (all-traces): verified (785 steps)
strict_registry_view_is_exact (all-traces): verified (25 steps)
no_cross_action_profile_or_audience_replay (all-traces): verified (41 steps)
execution_has_honest_approvals_or_prior_compromise (all-traces): verified (178 steps)
injective_execution_with_consumption (all-traces): verified (250 steps)
unchecked_composition_is_injective (all-traces): falsified - found trace (31 steps)
unchecked_registry_view_is_current (all-traces): falsified - found trace (20 steps)
```

The two falsified lemmas are deliberately weakened comparisons. Without consumption, the prover produces a same-receipt replay. Without exact registry-head binding, it produces acceptance under a stale or equivocating authority view. The strict paths verify, demonstrating that both checks are load-bearing.

The fourth model, `ep_six_claim_composed.spthy`, narrows the second composed path to six enforcement claims and two companion sanity properties. Its machine-checked summary is:

```
executable_six_claim_composition (exists-trace): verified (11 steps)
signed_denial_remains_verifiable_evidence (exists-trace): verified (8 steps)
class_a_downgrade_refused (all-traces): verified (22 steps)
signed_denial_cannot_authorize (all-traces): verified (2 steps)
scoped_authority_is_pinned (all-traces): verified (49 steps)
reliance_requires_pinned_profile (all-traces): verified (12 steps)
evidence_challenge_is_registered_and_consumed (all-traces): verified (41 steps)
fresh_challenge_registration_is_unique (all-traces): verified (2 steps)
aec_execution_is_action_keyed_and_fleet_fail_closed (all-traces): verified (13 steps)
action_reservation_failure_is_fail_closed (all-traces): verified (3 steps)
unchecked_presenter_class_is_pinned (all-traces): falsified - found trace (14 steps)
unchecked_signed_denial_cannot_authorize (all-traces): falsified - found trace (14 steps)
unchecked_authority_scope_is_pinned (all-traces): falsified - found trace (16 steps)
```

```
unchecked_reliance_profile_is_pinned (all-traces): falsified - found trace (14 steps)
unchecked_unregistered_challenge_is_registered (all-traces): falsified - found trace (14 ste
unchecked_presenter_execution_key_is_canonical (all-traces): falsified - found trace (14 ste
```

Each unsafe comparison removes one named restriction and produces the expected attack trace. The challenge result is intentionally narrow: it proves registration-before-reliance and one-time symbolic consumption, not HTTP semantics or database durability. Those are separately exercised by bounded state exploration and executable restart/concurrency tests.

The honest scope statement matters as much as the lemma list. Across the four models, the analysis does **not** model, and therefore proves nothing about:

- **Approver-directory publication**, **Merkle-log mechanics**, **checkpoints**, **inclusion proofs**, or transparency-log completeness. The composed model binds an exact pinned registry view as a symbolic value; it does not prove how that view was published or discovered.
- **WebAuthn internals**, authenticator hardware, or clientDataJSON parsing. User verification is an atomic gating event: the models assume the authenticator enforces user verification before releasing a signature. They do not prove WebAuthn.
- **Arbitrary k-of-n.** The quorum path fixes the 2-of-2 instance, the smallest case that exhibits distinctness and self-approval. The parametric k-of-n statement is not proven.
- **Collusion, coercion, and one-human-many-identities.** Separation of duties defeats unilateral self-approval and guarantees pairwise-distinct signing *identities*; it does not establish that two identities are two independent wills.
- **Canonicalization, amount arithmetic, policy authorship, or wall-clock freshness.** Canonical encoding is symbolic and injective; amount and policy semantics are abstract; no clock model is present. Those surfaces require parser, arithmetic, policy-governance, and freshness evidence outside this proof.
- **Computational or algorithm-specific security, and downstream exactly-once effects.** `sign` is the abstract Dolev-Yao signature with perfect cryptography; no reduction is offered for ES256, Ed25519, or ML-DSA. Consumption is proven within the modeled acceptance path, not across an arbitrary downstream side-effect system.

Symbolic verification does not prove the current validity or business correctness of any real receipt; it proves properties of the construction under these abstractions.

7.5 Complementary state-machine and relational model checking

The symbolic analysis above is about the protocol under a network attacker. Separately, and complementarily, the surrounding state machine is exhaustively model-checked at the state-machine level, where signature validity is *assumed*

and the object of study is the reachable state space rather than an active attacker.

The core state machine is a TLA+ model checked with TLC 2.19. The checked configuration explores the handshake lifecycle, signoff flow, delegation, revocation and consumption races, and identity-continuity extensions, generating 413,137 states (45,342 distinct) with no counterexample against 26 invariants. These include the properties an evidence artifact stands on: an authorization is never consumed twice; a consumed authorization is never subsequently revoked, and a revoked one never consumed; the lifecycle is acyclic and terminal states are inescapable; nonces are unique; delegation chains are acyclic and delegates cannot exceed their principals' authority; direct-write bypass of the state machine is impossible; concurrent revoke and consume interleavings serialize; and a denied action can never become approved. Model checking earned its keep during development: TLC surfaced four real specification bugs, all fixed before the properties passed. The checked configuration is a single handshake and a single claim, which covers all per-handshake and per-claim safety properties exhaustively within those bounds; it is not a general proof for unbounded concurrent instances, and the two-handshake configuration was bounded-tested (millions of distinct states, no counterexample) rather than exhausted. Exhaustively proving the composed, concurrent case is exactly the kind of question better suited to the symbolic prover than to TLC brute force, which is one reason the Tamarin models exist.

The quorum construction and the protocol's relational structure are separately checked in Alloy (6.1.0, SAT4J). One model proves fifteen assertions over the receipt and signoff relations (no double consumption, revoked-never-consumed, consumed-was-verified, terminal-state integrity, write-path exclusivity, delegation scope containment and acyclicity, signoff binding integrity and consume-once, and full-chain integrity). A federation model proves seven assertions about the cross-operator verification path (an accepted receipt is signed by an advertised key over an untampered payload, a tampered or unadvertised-key or revoked receipt is never accepted, historical keys still verify, no trust laundering, and observer-independent portability). These relational checks treat Ed25519 unforgeability as an abstract fact; they are structural, not cryptographic, which is precisely the gap the Tamarin models close.

The division of labor is deliberate. Alloy and TLA+ answer "does the state machine keep its invariants across the reachable space, assuming signatures behave"; Tamarin answers "do the security properties survive an attacker who controls the network and can solicit honest signatures, with signature validity derived rather than assumed." The three are complementary, and the paper does not present the state-machine model checking as a substitute for the protocol proof or the reverse.

7.6 Specified versus verified, and residual normative gaps

The distinction the project's own status tracking draws is the right one to repeat here: *specified* means a property is written down as a theorem; *verified* means a model checker has run and found no counterexample under the model's assumptions. The two are conflated in this field often enough that keeping them separate is itself a contribution to hygiene.

Two consequences follow and are stated normatively. First, several parts of the specification are specified but not covered by any model: the WebAuthn challenge binding beyond the atomic user-verification assumption, the approver directory protocol, log checkpoints, and the optional initiator-attestation member are specified, not proven, and extending the models to them is tracked work. Second, a few normative mechanisms are ahead of the reference implementation and not yet exercised by it or by conformance vectors, notably the operator-signed-directory assurance downgrade, delegation records in the receipt's key proofs together with the delegate-cannot-exceed-principal check, and enforcement-class emission; on these the specification is authoritative and the reference implementation is incomplete, not the reverse. Implementations must not represent any model as covering deployment topologies or components it does not model.

8. Implementation and Conformance

A reference implementation of the receipt, quorum, evidence-chain, binding, and evidence-graph layers is published under Apache-2.0. The conformance battery (21 suites, 332 vectors) is self-contained: each vector carries its own public key and document, requiring no server or shared state. The conformance battery of 21 suites comprising 332 vectors is adversarial (reject vectors each pin one invariant: tampered bytes, wrong key, replay, malformed signature, broken Merkle anchor) and deterministic (regenerable byte-identically from fixed seeds). Further suites cover quorum semantics, device signoffs, revocation timing, trusted-time attestation, the full trust receipt with Merkle inclusion and signed checkpoint (including a transparency-statement digest profile), provenance chains, evidence records, a canonicalization-malleability battery, the offline-verification boundary, currency (authentic-as-of-commit versus current validity), initiator-attestation field validation, sparse-Merkle one-time-consumption transitions, witness cosignatures over a checkpoint head, and an independent RFC 3161 timestamp-proof profile.

The implementation status is stated precisely, because it is the most tempting place to overclaim. The JavaScript, Python, and Go reference verifiers live in *one repository*: they are same-team ports whose agreement across all 332 current vectors is cross-language consistency evidence, not three independent implementations. A historical Ed25519-signed `EP-EXTERNAL-VERIFICATION-STATEMENT-v1` from COSA / J Diesel NY records external reproduction of 158 vectors against a pinned commit using this

project's own `emilia-verify` package; it remains exactly that historical reproduction result.

Separately, a Rust verifier authored by J Diesel NY is rebuilt by CI from immutable public source commit `7faba36010e7590727bebbc5b9dcceee60539b9b` and tree `0553c5fa0a5c4b566703e0d8ef9864dc33e5176d`. The evaluator runs it over the pinned 16-suite/164-vector clean-room bundle and a pinned hostility campaign of 353 structured attacks plus 6 raw-parser refusals, with zero divergences. The aggregate conformance case binds both campaigns to the same runner bytes. This is external interoperability and parser-robustness evidence. Its construction statement is signed by the implementation organization and predates the pinned source, not by a distinct attestor over the current source; consequently `EP-CONFORMANCE-CASE-v1` structurally reports zero strict independently attested clean-room acceptances. We do not upgrade that evidence into a construction-independence claim, and neither should a reader.

The evidence-graph layer ships with a test suite covering the fail-closed invariants of Section 6, including disclosure-independent graph identity, unbacked-edge poisoning, required-edge stripping, freshness degradation, and replay determinism, together with a complete worked vector (graph, relying-party policy, and signed reliance result) reproducible from a fixed seed. Deterministic vectors for the binding profile's checks and for the observed-absence statement are published alongside.

9. Related Work

The receipt is positioned as a composition with adjacent layers, not a replacement for any of them; each answers a different question, and the composition is the point.

Transparency and logging. SCITT (RFC 9943) provides an append-only transparency log with inclusion receipts, and is deliberately agnostic about *who authorized* a statement, which is precisely the question the authorization receipt answers. The two compose directly: a receipt can be expressed as a COSE signed statement and registered in a transparency service, and the binding profile's statement-digest form (Section 5) is designed for that carriage. The agent-action statement cluster emerging around SCITT (capsules, permits, post-execution records, refusal events) makes agent actions transparent, logged, and policy-checked; those profiles reserve slots for, and do not produce, the enrolled approver's action-bound approval evidence itself. Receiver-attested logging has the receiving service sign what it observed, post hoc; pre-execution authorization and post-execution attestation are complementary halves of a complete record.

Workload and agent identity. WIMSE workload credentials authenticate which workload is calling and prove key possession; their chartered scope is workload identity, and they deliberately do not define personal identities. A workload token is nonetheless carry-capable for the binding profile's reference

form, and the two answers have different trust roots (a workload key versus an enrolled approver key under organizational authority), which is why neither format can absorb the other. RATS (RFC 9334) and the Entity Attestation Token (RFC 9711) attest platform or workload trustworthiness, an orthogonal trust root; the receipt borrows EAT's verifier-relevant-nonce freshness discipline with a higher floor.

Authorization and delegation. OAuth Rich Authorization Requests (RFC 9396) and GNAP (RFC 9635) authorize a client's requested scope; step-up authentication (RFC 9470) can demand fresh human authentication but yields no durable, portable artifact of the approval. Transaction tokens (draft-ietf-oauth-transaction-tokens) propagate context across a machine call chain within a trust domain, short-lived and online-validated; the receipt is the human-approval evidence root from which such a chain can descend. Delegation-receipt work (draft-nelson-agent-delegation-receipts) binds a *user's* delegation to an *operator's* instructions, upstream; the authorization receipt binds an enrolled organizational approver key to an exact action downstream, with separation-of-duties and quorum semantics the delegation layer does not formalize, and the two compose by reference. CIBA may serve as the transport by which an approver is reached; the signoff is what the approver produces on arrival. Security Event Tokens (RFC 8417) and CAEP convey issuer assertions that an event occurred, not an enrolled approver key's pre-execution signoff bound to one action. Per-hop machine-side route and execution receipts govern delegated machine authority; none binds an enrolled approver key to a specific action under a human-approval policy, and the receipt's delegation constraints share their tighten-only scope-narrowing discipline. For long-lived evidence, Evidence Record Syntax (RFC 4998) supplies the re-timestamping pattern by which receipts remain verifiable after their original algorithms weaken.

Formal methods for authorization protocols. Symbolic protocol analysis under a Dolev-Yao attacker with tools such as Tamarin and ProVerif is a mature body of work; it has been applied to key-exchange and authentication protocols and, more recently, to WebAuthn and FIDO ceremonies. The contribution here is not the method but its application to action-bound human-approval evidence: deriving, rather than assuming, that acceptance of an authorization implies a prior user-verification-gated signature under a pinned enrolled key over exactly the accepted action, that quorum requires distinct enrolled identities with no self-approval, and that the one-time-consumption and initiator bindings the design depends on are the bindings that carry the properties. The complementary use of TLA+ for the state machine and Alloy for the relational structure follows their conventional strengths.

Separation from signature security. The result should not be confused with a new claim about the underlying signature scheme. Standard signature security addresses whether an adversary can produce a valid signature on a message that was never signed; it does not specify whether a verifier accepts that message twice, accepts it under a different action label, or counts the same identity in two

quorum slots. ABIA makes those verifier-side obligations explicit. The Tamarin falsifications are consequently separations between primitive authenticity and protocol authorization: the same ideal signing algebra is used in the safe and unsafe models, and only the omitted protocol binding changes.

10. Limitations

10.1 What a receipt does not prove

A receipt proves that a key enrolled under an approver identifier signed a canonical context binding one action and that the authorization reached a recorded terminal state within the relevant consumption domain. It does not prove that a particular natural person exercised the key: identifier-to-person binding is the directory and identity-proofing layer's property (Section 3.6), and possession of the enrolled device plus its user-verification factor is the practical proxy the artifact rests on. A coerced approver, a shoulder-surfed PIN, or a human who has enrolled multiple identities all produce receipts that verify. Distinctness checking operates over enrolled identities; whether two identities are two humans is an enrollment control, and the symbolic quorum model is explicit that it proves the distinct-identity property and nothing about independent wills (Section 7.4). Receipts make insider events attributable to enrolled identities and evidenced, which raises their cost; they do not make them impossible, and conforming implementations are prohibited from claiming otherwise.

10.2 Presentation attacks: narrowed, not solved

The gravest risk in the construction is that the approver signs context hash H believing it represents action X when it represents action Y. A signature proves user presence and an act of approval toward *whatever was rendered*; cryptography cannot prove the rendering was faithful, and the symbolic models do not attempt to: they assume the human signs the action term they are asked about. The specification requires escalating mitigations: the signing client must render from the exact bytes that were hashed, never from a separately supplied description; for high-value policies, render templates must be registered with the policy and committed under the policy hash, so display logic is inside what the signature covers; above an organization-designated threshold, material parameters must additionally be rendered on a second surface not authored by the orchestrating operator. Initiator-supplied free text is rendered as untrusted content under a hard length cap, visually separated from the operator-rendered action.

The residual risk is stated without minimization: absent a trusted display path, that is, hardware the operator does not author, rendering fidelity is enforced by controls, audit, and consented mismatch drills, not by mathematics. What the cryptography does provide is exactness of evidence: the receipt contains the full Action Object actually signed, so any divergence between what was displayed and what was executed is detectable after the fact with proof rather

than testimony. The evidence side of the what-you-see-is-what-you-sign problem is closed; the perception side is narrowed.

A related human-factors limitation is upstream of any attack: a gate that humans route around protects nothing, and rubber-stamping is the empirical failure mode of human-in-the-loop controls under volume. The construction is therefore not a general approval workflow; deployments are directed to scope signoff to genuinely high-risk, low-frequency actions and to treat signing-latency floors and zero deny rates as warning signs. The artifact's evidentiary value is only as strong as the attention of the human at its center.

10.3 Log completeness is unprovable offline

Offline verification establishes that a receipt was included in a log tree whose head the operator signed. It cannot establish, offline, that the tree shown to this verifier is the tree shown to everyone (equivocation requires gossip or witness cosigning to detect, which are online activities), nor that the log is complete: the absence of a receipt for some action is not offline-provable, and becomes evidence only through the signed observed-absence mechanism of Section 5, which attests a defined search and its emptiness, never a universal negative. For actions that a policy gates on signoff at an enforcing deployment class, the absence of any valid receipt is evidence that the control was bypassed; that property comes from the gate, not from the log. Revocation currency likewise requires an online consultation of a current directory head and checkpoint. These are boundaries of the offline model, stated as such, rather than defects a future version will quietly remove. The symbolic analysis inherits this boundary directly: it abstracts the log and directory as out-of-band pinning and proves nothing about split-view or completeness.

10.4 Scope of the sufficiency layer

An evidence-graph verdict is evidence of sufficiency under a stated, relying-party-owned policy at a stated time. It does not adjudicate disputes, does not establish business correctness, and inherits, without strengthening or weakening, the trust model of each artifact class it consumes. The publisher of these formats is not an auditor, regulator, or insurer; the artifacts are designed to support the determinations of such parties and never to substitute for them.

11. Conclusion

The gap this work addresses is narrow and, we have argued, real: the agent-security stack now produces abundant signed evidence about what an agent *is*, what it was *delegated*, what it *did*, and where that was *logged*, while evidence that an enrolled approver key signed a context binding one exact irreversible action before terminal consumption typically remains a database row in the custody of an interested party. The authorization receipt makes that narrower fact portable and offline-verifiable, with one-time terminal admission inside a

shared atomic consumption domain; the binding profile lets adjacent record formats carry it without redefining it; and the evidence-graph layer turns "is this pile of artifacts enough?" into a deterministic, replayable, relying-party-owned computation whose own outcome is evidence.

The design's discipline is subtraction. The operator is removed from the signature path, then prevented from re-entering through directory authority. The presenter is removed from the sufficiency decision. Symmetric primitives are removed from the verification path. Absence is removed from the space of things that can quietly count as consent. What remains is deliberately small: canonical bytes, approver-held keys, atomic terminal consumption, and proofs that travel inside the artifact. The symbolic analysis is what shows that small core actually carries its weight: under a network attacker with signature validity derived and not assumed, acceptance implies a prior user-verification-gated signature under the pinned enrolled key over exactly the accepted action, honest signatures cannot be transplanted, one-time consumption is what restores injectivity within the modeled domain, and a satisfied quorum requires distinct enrolled identities over the same action bound to the same initiator with no self-approval. The two falsification results, the replay that appears when consumption is dropped and the quorum break that appears when the initiator is not signed over, are the strongest evidence that these bindings are load-bearing rather than decorative.

We have been equally deliberate about the boundary of the claims. Two focused symbolic models cover the receipt core and a 2-of-2 quorum instance; two composed models analyze challenge, action identifier, approvals, authority, exact pinned reliance configuration, challenge consumption, and action-keyed fleet execution. They still exclude WebAuthn internals, directory publication and log mechanics, arbitrary k-of-n, canonical parsing and amount arithmetic, policy authorship, clock freshness, collusion, coercion, and downstream effects. The complementary TLA+ and Alloy models cover the surrounding state machine with signatures assumed. The cross-language ports are a consistency check, while the pinned external Rust verifier supplies separately authored interoperability and hostility evidence; its construction statement is implementer-signed, so strict independently attested acceptance remains zero. Rendering fidelity is enforced by controls rather than mathematics. And verification, everywhere in this design, proves signature, binding, and log-inclusion integrity as of commit time, never that the authorized action was correct. Evidence of authorization is not a verdict on conduct. It is the input that lets whoever must reach such a verdict, an auditor, an insurer, a court, do so from artifacts rather than testimony, and that is the entire ambition of the design.

Artifact Availability

The following are published under the Apache-2.0 license in the EMILIA Protocol repository: four Tamarin symbolic models (`ep_receipt_core.spthy`,

`ep_quorum_core.spthy`, `ep_reliance_composed.spthy`, and `ep_six_claim_composed.spthy`) with model headers, verbatim tool output, falsification history, and re-run instructions; the TLA+ and Alloy models with checked configurations and a proof-status ledger that keeps *specified* and *verified* separate; the reference implementation (receipt, quorum, evidence-chain, host-record binding, and evidence-graph layers, with JavaScript, Python, and Go same-team ports); the 21-suite / 332-vector conformance battery with deterministic regeneration scripts; the pinned external Rust source and evaluator; the 359-case hostility campaign; the aggregate conformance case; the historical third-party reproduction statement; and the worked vectors referenced in this paper. The construction is also specified, in engineering form, as individual Internet-Drafts via the IETF Datatracker (draft-schrock-ep-authorization-receipts and companion documents); Internet-Drafts are working documents and should be cited as work in progress.

References

- I. Schrock, "Authorization Receipts for High-Risk Agent Actions," Internet-Draft draft-schrock-ep-authorization-receipts-12, work in progress, August 2026.
- I. Schrock, "Multi-Party Human Authorization (EP-QUORUM)," Internet-Draft draft-schrock-ep-quorum-03, work in progress, July 2026.
- I. Schrock, "Authorization Evidence Chains: Composing Heterogeneous Agent-Action Evidence (EP-AEC)," Internet-Draft draft-schrock-ep-authorization-evidence-chain-05, work in progress, August 2026.
- I. Schrock, "Long-Term Evidence Records for Authorization Receipts (EP-EVIDENCE-RECORD)," Internet-Draft draft-schrock-ep-evidence-record-01, work in progress, July 2026.
- S. Meier, B. Schmidt, C. Cremers, D. Basin, "The TAMARIN Prover for the Symbolic Analysis of Security Protocols," CAV 2013.
- S. Goldwasser, S. Micali, R. Rivest, "A Digital Signature Scheme Secure Against Adaptive Chosen-Message Attacks," SIAM Journal on Computing 17(2), 1988.
- D. Dolev, A. C. Yao, "On the Security of Public Key Protocols," IEEE Transactions on Information Theory, 1983.
- L. Lamport, "Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers," Addison-Wesley, 2002.
- D. Jackson, "Software Abstractions: Logic, Language, and Analysis," MIT Press, revised edition, 2012.
- A. Rundgren, B. Jordan, S. Erdtman, "JSON Canonicalization Scheme (JCS)," RFC 8785, June 2020.
- W3C, "Web Authentication: An API for accessing Public Key Credentials, Level 2," April 2021.
- "An Architecture for Trustworthy and Transparent Digital Supply Chains (SCITT)," RFC 9943.
- J. Schaad, "CBOR Object Signing and Encryption (COSE): Structures

and Process," RFC 9052.
- "The Entity Attestation Token (EAT)," RFC 9711.
- "Remote ATtestation procedureS (RATS) Architecture," RFC 9334.
- "Grant Negotiation and Authorization Protocol (GNAP)," RFC 9635.
- "OAuth 2.0 Rich Authorization Requests," RFC 9396.
- "OAuth 2.0 Step Up Authentication Challenge Protocol," RFC 9470.
- "Security Event Token (SET)," RFC 8417.
- "Evidence Record Syntax (ERS)," RFC 4998.
- OpenID Foundation, "OpenID Connect Client-Initiated Backchannel Authentication Flow, Core 1.0," September 2021.
- R. Nelson, "Delegation Receipt Protocol for AI Agent Authorization," Internet-Draft draft-nelson-agent-delegation-receipts, work in progress, June 2026.
- "Transaction Tokens," Internet-Draft draft-ietf-oauth-transaction-tokens, work in progress.